Skill – 5

PHYSICIAN & DEPARTMENT TABLES

STEP 1: CREATE PHYSICIAN TABLE

```
CREATE TABLE physician
(     employeeid INT PRIMARY
KEY,     name
VARCHAR(100),position
VARCHAR(100),ssn VARCHAR(20)
);
```

```
mysql> CREATE TABLE physician (
    ->     employeeid INT PRIMARY KEY,
    ->     name VARCHAR(100),
    ->     position VARCHAR(100),
    ->     ssn VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.19 sec)
```

STEP 2: INSERT DATA INTO PHYSICIAN

```
INSERT INTO physician VALUES
(1,'John Dorian','Staff Internist','111111111'),
(2,'Elliot Reid','Attending Physician','222222222'),
(3,'Christopher Turk','Surgical Attending
Physician','333333333'),
(4,'Percival Cox','Senior Attending Physician','444444444'),
(5,'Bob Kelso','Head Chief of Medicine','555555555'),
(6,'Todd Quinlan','Surgical Attending Physician','666666666'),
(7,'John Wen','Surgical Attending Physician','777777777'),
(8,'Keith Dudemeister','MD Resident','888888888'),
(9,'Molly Clock','Attending Psychiatrist','999999999');
```

```
mysql> INSERT INTO physician VALUES
    -> (1,'John Dorian','Staff Internist','111111111'),
    -> (2,'Elliot Reid','Attending Physician','222222222'),
    -> (3,'Christopher Turk','Surgical Attending Physician','333333333'),
    -> (4,'Percival Cox','Senior Attending Physician','444444444'),
    -> (5,'Bob Kelso','Head Chief of Medicine','555555555'),
    -> (6,'Todd Quinlan','Surgical Attending Physician','666666666'),
    -> (7,'John Wen','Surgical Attending Physician','777777777'),
    -> (8,'Keith Dudemeister','MD Resident','888888888'),
    -> (9,'Molly Clock','Attending Psychiatrist','999999999');
Query OK, 9 rows affected (0.03 sec)
Records: 9  Duplicates: 0  Warnings: 0
```

STEP 3: CREATE DEPARTMENT TABLE

```
CREATE TABLE department
(     departmentid INT PRIMARY
KEY,     name VARCHAR(100),
head INT
);
```

```
mysql> CREATE TABLE department (
    ->     departmentid INT PRIMARY KEY,
    ->     name VARCHAR(100),
    ->     head INT
    -> );
Query OK, 0 rows affected (0.08 sec)
```

STEP 4: INSERT DATA INTO DEPARTMENT

```
INSERT INTO department VALUES
(1,'General Medicine',4),
(2,'Surgery',7),
(3,'Psychiatry',9);
```

```
mysql> INSERT INTO department VALUES
    -> (1,'General Medicine',4),
    -> (2,'Surgery',7),
    -> (3,'Psychiatry',9);
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

QUESTION 1 - INNER JOIN

SCENARIO:
The hospital administration wants a report showing each department and its department head.
QUERY:

```
SELECT d.name AS Department,
       p.name AS Head_Physician
FROM department d
JOIN physician p
ON d.head = p.employeeid;
```

```
mysql> SELECT d.name AS Department,
    ->        p.name AS Head_Physician
    -> FROM department d
    -> JOIN physician p
    -> ON d.head = p.employeeid;
+------------------+----------------+
| Department       | Head_Physician |
+------------------+----------------+
| General Medicine | Percival Cox   |
| Surgery          | John Wen       |
| Psychiatry       | Molly Clock    |
+------------------+----------------+
3 rows in set (0.01 sec)
```

QUESTION 2 - WHERE CLAUSE

SCENARIO:
The HR department wants a list of all physicians working as Surgical Attending Physicians.
QUERY:

```
SELECT *
FROM physician
WHERE position='Surgical Attending Physician';
```

```
mysql> SELECT *
    -> FROM physician
    -> WHERE position='Surgical Attending Physician';
+------------+------------------+-----------------------------+-----------+
| employeeid | name             | position                    | ssn       |
+------------+------------------+-----------------------------+-----------+
|          3 | Christopher Turk | Surgical Attending Physician | 333333333 |
|          6 | Todd Quinlan     | Surgical Attending Physician | 666666666 |
|          7 | John Wen         | Surgical Attending Physician | 777777777 |
+------------+------------------+-----------------------------+-----------+
3 rows in set (0.01 sec)
```

QUESTION 3 - WHERE + LIKE

SCENARIO:
Management wants to identify physicians whose names start with 'John'.
QUERY:

```
SELECT *
FROM physician
WHERE name LIKE 'John%';
```

```
mysql> SELECT *
    -> FROM physician
    -> WHERE name LIKE 'John%';
+------------+------------+-----------------------------+-----------+
| employeeid | name       | position                    | ssn       |
+------------+------------+-----------------------------+-----------+
|          1 | John Dorian | Staff Internist             | 111111111 |
|          7 | John Wen    | Surgical Attending Physician | 777777777 |
+------------+------------+-----------------------------+-----------+
2 rows in set (0.01 sec)
```

QUESTION 4 - COUNT()

SCENARIO:
The hospital wants to know the total number of physicians currently employed.
QUERY:

```
SELECT COUNT(*) AS Total_Physicians
FROM physician;
```

```
mysql> SELECT COUNT(*) AS Total_Physicians
    -> FROM physician;
+------------------+
| Total_Physicians |
+------------------+
|                9 |
+------------------+
1 row in set (0.03 sec)
```

QUESTION 5 - COUNT DISTINCT

SCENARIO:
HR wants to know how many different job positions exist in the hospital.
QUERY:

```
SELECT COUNT(DISTINCT position)
FROM physician;
```

```
mysql> SELECT COUNT(DISTINCT position)
    -> FROM physician;
+--------------------------+
| COUNT(DISTINCT position) |
+--------------------------+
|                        7 |
+--------------------------+
1 row in set (0.01 sec)
```

QUESTION 6 - GROUP BY

QUESTION 6 - GROUP BY

SCENARIO:
The HR manager wants to know how many physicians are working in each position.

QUERY:

```
SELECT position,
       COUNT(*) AS Total_Employees
FROM physician
GROUP BY position;
```

```
mysql> SELECT position,
    ->        COUNT(*) AS Total_Employees
    -> FROM physician
    -> GROUP BY position;
+-----------------------------+-----------------+
| position                    | Total_Employees |
+-----------------------------+-----------------+
| Staff Internist             |               1 |
| Attending Physician         |               1 |
| Surgical Attending Physician |               3 |
| Senior Attending Physician  |               1 |
| Head Chief of Medicine      |               1 |
| MD Resident                 |               1 |
| Attending Psychiatrist      |               1 |
+-----------------------------+-----------------+
7 rows in set (0.01 sec)
```

QUESTION 7 - GROUP BY + HAVING

SCENARIO:
The HR team wants to find positions occupied by more than one physician.

QUERY:

```
SELECT position,
       COUNT(*) AS Total_Employees
FROM physician
GROUP BY position
HAVING COUNT(*) > 1;
```

```
mysql> SELECT position,
    ->        COUNT(*) AS Total_Employees
    -> FROM physician
    -> GROUP BY position
    -> HAVING COUNT(*) > 1;
+------------------------------+-----------------+
| position                     | Total_Employees |
+------------------------------+-----------------+
| Surgical Attending Physician |               3 |
+------------------------------+-----------------+
1 row in set (0.03 sec)
```

QUESTION 8 - ORDER BY

SCENARIO:
Hospital management wants an alphabetical directory of physicians.

QUERY:

```
SELECT *
FROM physician
ORDER BY name ASC;
```

```
mysql> SELECT *
    -> FROM physician
    -> ORDER BY name ASC;
+------------+------------------+------------------------------+-----------+
| employeeid | name             | position                     | ssn       |
+------------+------------------+------------------------------+-----------+
|          5 | Bob Kelso        | Head Chief of Medicine       | 555555555 |
|          3 | Christopher Turk | Surgical Attending Physician | 333333333 |
|          2 | Elliot Reid      | Attending Physician          | 222222222 |
|          1 | John Dorian      | Staff Internist              | 111111111 |
|          7 | John Wen         | Surgical Attending Physician | 777777777 |
|          8 | Keith Dudemeister | MD Resident                 | 888888888 |
|          9 | Molly Clock      | Attending Psychiatrist       | 999999999 |
|          4 | Percival Cox     | Senior Attending Physician   | 444444444 |
|          6 | Todd Quinlan     | Surgical Attending Physician | 666666666 |
+------------+------------------+------------------------------+-----------+
9 rows in set (0.02 sec)
```

QUESTION 9 - ORDER BY DESC

SCENARIO:
Management wants to see physician records sorted by employee ID from highest to lowest.

QUERY:

```
SELECT *
FROM physician
```

```
ORDER BY employeeid DESC;
```

```
mysql> SELECT *
    -> FROM physician
    -> ORDER BY employeeid DESC;
+------------+------------------+-----------------------------+-----------+
| employeeid | name             | position                    | ssn       |
+------------+------------------+-----------------------------+-----------+
|          9 | Molly Clock      | Attending Psychiatrist      | 999999999 |
|          8 | Keith Dudemeister | MD Resident                | 888888888 |
|          7 | John Wen         | Surgical Attending Physician | 777777777 |
|          6 | Todd Quinlan     | Surgical Attending Physician | 666666666 |
|          5 | Bob Kelso        | Head Chief of Medicine      | 555555555 |
|          4 | Percival Cox     | Senior Attending Physician  | 444444444 |
|          3 | Christopher Turk | Surgical Attending Physician | 333333333 |
|          2 | Elliot Reid      | Attending Physician         | 222222222 |
|          1 | John Dorian      | Staff Internist             | 111111111 |
+------------+------------------+-----------------------------+-----------+
9 rows in set (0.01 sec)
```

QUESTION 10 - WHERE + IN

SCENARIO:
The CEO wants details of department heads only.

QUERY:

```
SELECT *
FROM physician
WHERE employeeid IN
(
    SELECT head
    FROM department
);
```

```
mysql> SELECT *
    -> FROM physician
    -> WHERE employeeid IN
    -> (
    ->     SELECT head
    ->     FROM department
    -> );
+------------+-------------+-----------------------------+-----------+
| employeeid | name        | position                    | ssn       |
+------------+-------------+-----------------------------+-----------+
|          4 | Percival Cox | Senior Attending Physician  | 444444444 |
|          7 | John Wen    | Surgical Attending Physician | 777777777 |
|          9 | Molly Clock | Attending Psychiatrist      | 999999999 |
+------------+-------------+-----------------------------+-----------+
3 rows in set (0.05 sec)
```

QUESTION 11 – SUBQUERY

SCENARIO:
Find physicians who are not department heads.

QUERY:

```
SELECT *
FROM physician
WHERE employeeid NOT IN
(
    SELECT head
    FROM department
);
```

```
mysql> SELECT *
    -> FROM physician
    -> WHERE employeeid NOT IN
    -> (
    ->     SELECT head
    ->     FROM department
    -> );
+------------+------------------+-----------------------------+-----------+
| employeeid | name             | position                    | ssn       |
+------------+------------------+-----------------------------+-----------+
|          1 | John Dorian      | Staff Internist             | 111111111 |
|          2 | Elliot Reid      | Attending Physician         | 222222222 |
|          3 | Christopher Turk | Surgical Attending Physician | 333333333 |
|          5 | Bob Kelso        | Head Chief of Medicine      | 555555555 |
|          6 | Todd Quinlan     | Surgical Attending Physician | 666666666 |
|          8 | Keith Dudemeister | MD Resident                | 888888888 |
+------------+------------------+-----------------------------+-----------+
6 rows in set (0.02 sec)
```

QUESTION 12 – GROUP BY + ORDER BY

SCENARIO:
HR wants to see positions ranked by number
of employees.

QUERY:

```
SELECT position,
       COUNT(*) AS Total_Employees
FROM physician
GROUP BY position
ORDER BY Total_Employees DESC;
```

```
mysql> SELECT position,
    ->        COUNT(*) AS Total_Employees
    -> FROM physician
    -> GROUP BY position
    -> ORDER BY Total_Employees DESC;
+-----------------------------+-----------------+
| position                    | Total_Employees |
+-----------------------------+-----------------+
| Surgical Attending Physician |               3 |
| Staff Internist             |               1 |
| Attending Physician         |               1 |
| Senior Attending Physician  |               1 |
| Head Chief of Medicine      |               1 |
| MD Resident                 |               1 |
| Attending Psychiatrist      |               1 |
+-----------------------------+-----------------+
7 rows in set (0.00 sec)
```

QUESTION 13 - WHERE + LIKE

SCENARIO:
The administration wants all physicians whose position contains the word 'Attending'.

QUERY:

```
SELECT *
FROM physician
WHERE position LIKE '%Attending%';
```

```
mysql> SELECT *
    -> FROM physician
    -> WHERE position LIKE '%Attending%';
+------------+-----------------+-----------------------------+-----------+
| employeeid | name            | position                    | ssn       |
+------------+-----------------+-----------------------------+-----------+
|          2 | Elliot Reid     | Attending Physician         | 222222222 |
|          3 | Christopher Turk | Surgical Attending Physician | 333333333 |
|          4 | Percival Cox    | Senior Attending Physician  | 444444444 |
|          6 | Todd Quinlan    | Surgical Attending Physician | 666666666 |
|          7 | John Wen        | Surgical Attending Physician | 777777777 |
|          9 | Molly Clock     | Attending Psychiatrist      | 999999999 |
+------------+-----------------+-----------------------------+-----------+
6 rows in set (0.02 sec)
```

QUESTION 14 - JOIN + WHERE

SCENARIO:
The hospital wants to know who heads the Surgery department.

QUERY:

```
SELECT p.name
FROM physician p
JOIN department d
ON p.employeeid=d.head
WHERE d.name='Surgery';
```

```
mysql> SELECT p.name
    -> FROM physician p
    -> JOIN department d
    -> ON p.employeeid=d.head
    -> WHERE d.name='Surgery';
+----------+
| name     |
+----------+
| John Wen |
+----------+
1 row in set (0.02 sec)
```

QUESTION 15 - JOIN + ORDER BY

```
SCENARIO:
Hospital administration wants department names
and head physicians sorted by department name.

QUERY:

SELECT d.name,
       p.name
FROM department d
JOIN physician p
ON d.head=p.employeeid
ORDER BY d.name;
```

```
mysql> SELECT d.name,
    ->        p.name
    -> FROM department d
    -> JOIN physician p
    -> ON d.head=p.employeeid
    -> ORDER BY d.name;
+------------------+--------------+
| name             | name         |
+------------------+--------------+
| General Medicine | Percival Cox |
| Psychiatry       | Molly Clock  |
| Surgery          | John Wen     |
+------------------+--------------+
3 rows in set (0.00 sec)
```

QUESTION 16 - WHERE + NOT LIKE

SCENARIO:
Find physicians who are not involved in surgery.

QUERY:

```
SELECT *
FROM physician
WHERE position NOT LIKE '%Surgical%';
```

```
mysql> SELECT *
    -> FROM physician
    -> WHERE position NOT LIKE '%Surgical%';
+------------+------------------+---------------------------+-----------+
| employeeid | name             | position                  | ssn       |
+------------+------------------+---------------------------+-----------+
|          1 | John Dorian      | Staff Internist           | 111111111 |
|          2 | Elliot Reid      | Attending Physician       | 222222222 |
|          4 | Percival Cox     | Senior Attending Physician | 444444444 |
|          5 | Bob Kelso        | Head Chief of Medicine    | 555555555 |
|          8 | Keith Dudemeister | MD Resident              | 888888888 |
|          9 | Molly Clock      | Attending Psychiatrist    | 999999999 |
+------------+------------------+---------------------------+-----------+
6 rows in set (0.03 sec)
```

QUESTION 17 - COUNT + HAVING

SCENARIO:
Find positions having at least 2 employees.

QUERY:

```
SELECT position,
       COUNT(*) AS Total
FROM physician
GROUP BY position
HAVING COUNT(*) >= 2;
```

```
mysql> SELECT position,
    ->        COUNT(*) AS Total
    -> FROM physician
    -> GROUP BY position
    -> HAVING COUNT(*) >= 2;
+----------------------------+-------+
| position                   | Total |
+----------------------------+-------+
| Surgical Attending Physician |     3 |
+----------------------------+-------+
1 row in set (0.01 sec)
```

QUESTION 18 - EXISTS

SCENARIO:
Find departments that currently have a valid head.

QUERY:

```
SELECT *
FROM department d
WHERE EXISTS
(
    SELECT 1
    FROM physician p
    WHERE p.employeeid=d.head
);
```

```
mysql> SELECT *
    -> FROM department d
    -> WHERE EXISTS
    -> (
    ->     SELECT 1
    ->     FROM physician p
    ->     WHERE p.employeeid=d.head
    -> );
+--------------+------------------+------+
| departmentid | name             | head |
+--------------+------------------+------+
|            1 | General Medicine |    4 |
|            2 | Surgery          |    7 |
|            3 | Psychiatry       |    9 |
+--------------+------------------+------+
3 rows in set (0.00 sec)
```

QUESTION 19 - MAX()

SCENARIO:
Management wants the highest employee ID
currently assigned.

QUERY:

```
SELECT MAX(employeeid)
FROM physician;
```

```
mysql> SELECT MAX(employeeid)
    -> FROM physician;
+-----------------+
| MAX(employeeid) |
+-----------------+
|               9 |
+-----------------+
1 row in set (0.01 sec)
```

QUESTION 20 - MIN()

SCENARIO:
Management wants the earliest employee ID currently assigned.

QUERY:

```
SELECT MIN(employeeid)
FROM physician;
```

```
mysql> SELECT MIN(employeeid)
    -> FROM physician;
+-----------------+
| MIN(employeeid) |
+-----------------+
|               1 |
+-----------------+
1 row in set (0.00 sec)
```